

AutoJudge: Predicting Programming Problem Difficulty

Prateek Dixit

BS-MS (Economics), IIT Roorkee

❖ Introduction

Online competitive programming platforms such as Codeforces, CodeChef, and Kattis host a large number of programming problems spanning a wide range of difficulty levels. These problems are typically labeled using discrete difficulty categories (Easy, Medium, Hard) and, in some cases, a numerical difficulty score.

Manual assignment of problem difficulty is subjective and often inconsistent across platforms. It also requires significant human effort. With the increasing availability of programming problem data, there is strong motivation to automate difficulty prediction using machine learning techniques.

This project, AutoJudge, aims to automatically predict:

- The difficulty class (Easy / Medium / Hard)
- A numerical difficulty score

using only the textual content of programming problems. No metadata such as submission statistics, solution code, or user interaction data is used.

❖ Problem Statement

The objective of this project is to design and implement a machine learning system capable of:

1. Classification Task: Predicting the difficulty category of a programming problem (Easy / Medium / Hard).
2. Regression Task: Predicting a numerical difficulty score on a continuous scale.

The predictions are based solely on:

- Problem Description
- Input Description
- Output Description

This makes the task challenging, as difficulty must be inferred indirectly from linguistic cues, algorithmic hints, constraints, and structural complexity present in the text.

❖ Dataset Description

The dataset consists of programming problems with labeled difficulty information. Each data sample includes the following fields:

- description
- input_description
- output_description
- problem_class (Easy / Medium / Hard)
- problem_score (numerical value between 1 and 10)

The dataset did not contain missing values in the relevant fields. Irrelevant columns such as problem URLs and sample I/O were removed prior to modeling.

❖ Data Preprocessing

@Text Combination

To represent the complete problem statement, the following fields were concatenated into a single text input:

- Problem Description
- Input Description
- Output Description

This combined text was used consistently for both training and inference.

@Text Cleaning

Standard NLP preprocessing steps were applied:

- Conversion to lowercase
- Removal of extra whitespace
- Normalization of textual content

These steps helped reduce noise and ensured consistent feature extraction.

❖ Feature Engineering

@TF-IDF Features

Term Frequency–Inverse Document Frequency (TF-IDF) was used to convert text into numerical features:

- Unigrams and bigrams
- Stop-word removal
- High-dimensional sparse representation

TF-IDF captures important keywords and phrases that often indicate algorithmic complexity (e.g., “constraints”, “graph”, “dynamic programming”).

@Dimensionality Reduction (Classification)

For classification models, **Truncated SVD** was applied to TF-IDF vectors to reduce dimensionality while preserving semantic structure. This also improved training stability and inference speed.

❖ Models Used

@Classification Models

The following classification models were evaluated:

1. Logistic Regression (TF-IDF + SVD)
2. Random Forest Classifier (TF-IDF + SVD)
3. Linear Support Vector Machine (Sentence Transformer embeddings)

Each model was evaluated using accuracy, confusion matrices, and **Macro F1 score**, which was emphasized due to class imbalance.

@Regression Model

The following regression models were evaluated:

- Ridge Regression
- Random Forest Regressor
- Gradient Boosting Regressor (GBR)

Performance was measured using **Mean Absolute Error (MAE)** and **Root Mean Squared Error (RMSE)**.

❖ Experimental Setup

- Train-test split: **80% training, 20% testing**
 - Stratified sampling used for classification
 - Evaluation metrics:
 - Accuracy
 - Confusion Matrix
 - Macro F1 Score (classification)
 - MAE and RMSE (regression)
-

❖ Evaluation Results

@Classification Results

Logistic Regression (TF-IDF + SVD)

- **Overall Accuracy: 0.464**
- **Macro F1 Score: 0.448**

This model showed balanced performance across classes and achieved the **highest Macro F1 score**, making it the most reliable classifier.

```
Logistic Regression - Confusion Matrix
[[ 80  39  17]
 [ 70  92 100]
 [ 80 135 210]]

Class-wise Accuracy (Logistic Regression):
easy: 0.588
medium: 0.351
hard: 0.494

Overall Accuracy (Logistic Regression): 0.4641555285540705
```

Random Forest Classifier (TF-IDF + SVD)

- **Overall Accuracy: 0.512**
- **Macro F1 Score: 0.362**

Although Random Forest achieved higher accuracy, It heavily favored the majority “Hard” class and performed poorly on Easy and Medium classes, leading to a significantly lower Macro F1 score.

```
** Random Forest - Confusion Matrix
[[ 20  30  86]
 [ 11  35 216]
 [  5  54 366]]

***
Class-wise Accuracy (Random Forest):
easy: 0.147
medium: 0.134
hard: 0.861

Overall Accuracy (Random Forest): 0.511543134872418
```

```
RF Macro F1: 0.36200051686660295
LR Macro F1: 0.4480513189093586
```

Sentence Transformer + Linear SVM

- **Overall Accuracy: 0.498**
- **Macro F1 Score: 0.469**

Accuracy: 0.49817739975698666				
	precision	recall	f1-score	support
easy	0.39	0.51	0.44	136
hard	0.64	0.58	0.61	425
medium	0.36	0.35	0.36	262
accuracy			0.50	823
macro avg	0.46	0.48	0.47	823
weighted avg	0.51	0.50	0.50	823

This model achieved strong semantic understanding but struggled to separate Medium-level problems clearly. Due to higher computational cost and marginal gains, it was not selected for deployment.

@Model Selection Justification

Despite Random Forest achieving higher accuracy, **Logistic Regression was chosen as the final classification model** due to its:

- Higher Macro F1 score
- More balanced class-wise performance
- Better generalization behavior

Accuracy alone was found to be misleading due to class imbalance.

@Regression Results (Difficulty Score Prediction)

Ridge Regression

Ridge Regression was chosen as a baseline linear regression model due to its stability and effectiveness in high-dimensional feature spaces such as TF-IDF.

```
MAE: 1.771560033461539
RMSE: 2.1113056696700263
```

Ridge Regression provided consistent predictions and demonstrated good generalization performance, making it suitable for deployment.

Random Forest Regressor

Random Forest Regression was evaluated to capture potential non-linear relationships between textual features and difficulty scores.

```
MAE : 1.7732059279493884
RMSE: 2.100638930437112
```

While Random Forest achieved slightly lower RMSE compared to Ridge Regression, the improvement was marginal. Additionally, Random Forest models are more complex and less interpretable.

Gradient Boosting Regressor (GBR)

Gradient Boosting Regression was also tested, as it is known to perform well on structured regression tasks by sequentially minimizing prediction errors.

```
MAE : 1.7560140570261413
RMSE: 2.0906863204141
```

GBR achieved the **lowest MAE and RMSE** among the tested regression models, indicating slightly better predictive accuracy.

Regression Model Comparison

Model	MAE	RMSE
Ridge Regression	1.77	2.11
Random Forest Regressor	1.77	2.10
Gradient Boosting Regressor	1.76	2.09

Although Gradient Boosting Regressor achieved the best numerical performance, the differences between the models were relatively small.

Final Regression Model Selection

For deployment in the web application, **Ridge Regression** was selected despite Gradient Boosting Regressor achieving slightly better error metrics. This decision was motivated by:

- Lower computational complexity
- Faster inference time
- Better stability on high-dimensional sparse features
- Easier interpretability

Given the inherent subjectivity and noise in textual difficulty estimation, the marginal improvement offered by ensemble-based models did not justify the added complexity for deployment.

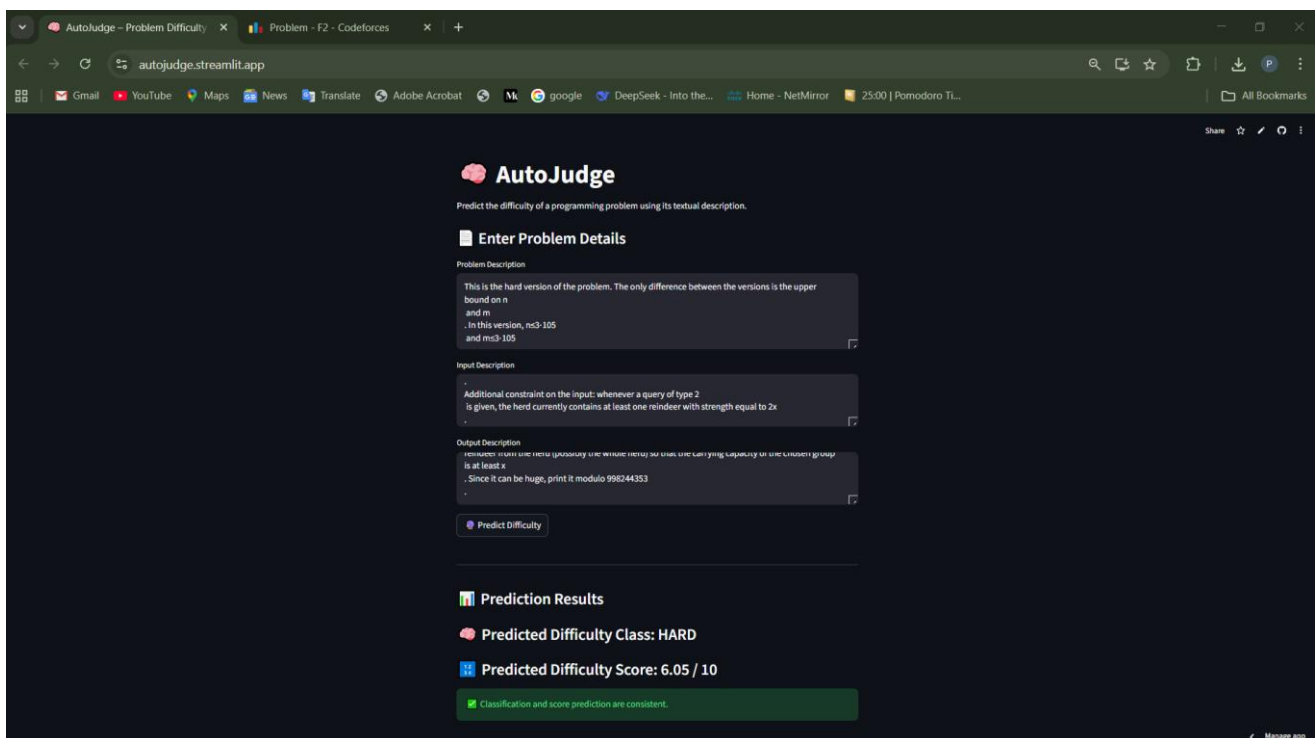
❖ Web Interface

A **Streamlit-based web application** was developed to demonstrate the system.

Workflow:

1. User enters:
 - Problem Description
 - Input Description
 - Output Description
2. User clicks **Predict Difficulty**
3. The application:
 - Combines text inputs
 - Applies trained models
4. Displays:
 - Predicted difficulty class
 - Predicted difficulty score
 - A warning for borderline cases when predictions differ

IMAGE [below is the prediction of a hard codeforces problem (EDU- PROBLEM F2)]



❖ Conclusion

This project demonstrates that programming problem difficulty can be reasonably predicted using only textual information. While perfect accuracy is not expected due to subjectivity, the models capture meaningful patterns related to algorithmic complexity and constraints.

Key takeaways:

- Macro F1 is a more reliable metric than accuracy for imbalanced difficulty classification.
- Linear models with proper feature engineering can outperform complex models in fairness and stability.
- Medium-level problems remain the most challenging due to overlapping characteristics.

❖ Future Work

- Incorporating transformer-based embeddings more deeply
- Using hierarchical classification strategies
- Platform-specific difficulty modeling
- Incorporating constraint parsing explicitly